

Java Backend Architecture

Interview Series

Chapter 19.7

GCP Core Services for Java

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 19.7 – GCP Core Services for Java

Overview

Google Cloud Platform offers GKE, Cloud Run, and Cloud SQL as the primary compute and data services for Java backend deployments. GKE's Autopilot mode and Cloud Run's serverless containers significantly reduce operational overhead. Workload Identity eliminates credential management. Understanding these services enables informed GCP vs AWS architectural decisions.

Key Definitions

Term	Definition
GKE Standard	GKE with self-managed node pools. Full control over node types, OS, autoscaling policies.
GKE Autopilot	GKE where Google manages nodes. Pod-level billing, hardened security, no node SSH. Hands-off.
Workload Identity	GCP IAM equivalent of AWS IRSA. Kubernetes ServiceAccount maps to GCP Service Account.
Cloud Run	Fully managed serverless containers. HTTP-triggered, auto-scales to 0, per-request billing.
Cloud Run Jobs	Cloud Run for batch workloads. Run to completion, parallelism, no HTTP trigger required.
Cloud SQL	Managed PostgreSQL/MySQL. HA with standby in different zone. Read replicas. Auth Proxy for IAM auth.
Cloud SQL Auth Proxy	Sidecar that handles IAM authentication to Cloud SQL. No IP allowlisting, encrypted tunnel.
Binary Authorization	GKE policy requiring all images to be signed by trusted parties before deployment.
Cloud Armor	GCP WAF + DDoS protection. Attaches to Google Cloud Load Balancer. Geo-based blocking.
Artifact Registry	GCP managed container/package registry. Replaces Container Registry. IAM-controlled access.

Diagram 1: GKE Autopilot vs Standard

GKE Standard:	GKE Autopilot:
You manage:	Google manages:
- Node pool configuration	- Node provisioning
- Machine types	- OS patching
- OS patching	- Node auto-repair
- Node auto-repair	
You control:	You specify only:
- Node SSH access	- Pod CPU/memory requests
- Custom node configs	- (Google selects machine type)
- Spot/preemptible nodes	
Billing: per node (EC2 equiv)	Billing: per pod CPU+memory (not per node)
	Enforce: no privileged containers auto (hardened by default)

Diagram 2: Cloud Run Scale-to-Zero

Traffic Pattern:

Requests:												
Instances:	0	1	2	1	0		5-10		0	1	0	

Spring Boot on Cloud Run:

- Idle > 15 min: scale to 0 (no cost)
- First request after idle: cold start (JVM init + Spring context)
- min-instances=1: keep warm, eliminates cold start (small cost)
- max-instances=100: auto-scale on load
- concurrency=80: requests per container (HTTP/2 or Java thread pool)

Code Examples

GKE Autopilot + Workload Identity:

```
# 1. Create GKE Autopilot cluster
gcloud container clusters create-auto java-cluster --region us-central1 --workload-pool=PROJECT_ID.svc.id.goog

# 2. Create GCP Service Account
gcloud iam service-accounts create java-app-sa
gcloud projects add-iam-policy-binding PROJECT_ID --member="serviceAccount:java-app-sa@PROJECT_ID.iam.gserviceaccount.com" --role="roles/storage.objectViewer"

# 3. Bind K8s SA to GCP SA
gcloud iam service-accounts add-iam-policy-binding java-app-sa@PROJECT_ID.iam.gserviceaccount.com --role roles/iam.workloadIdentityUser --member "serviceAccount:PROJECT_ID.svc.id.goog[default/java-k8s-sa]"

# 4. Annotate K8s ServiceAccount
kubectl annotate serviceaccount java-k8s-sa iam.gke.io/gcp-service-account=java-app-sa@PROJECT_ID.iam.gserviceaccount.com
```

Cloud Run Java Deployment:

```
# Build and deploy Spring Boot to Cloud Run
gcloud run deploy java-api --image gcr.io/PROJECT_ID/java-api:latest --region us-central1 --platform managed --min-instances 1 --max-instances 100 --concurrency 80 --memory 1Gi --cpu 1 --set-env-vars SPRING_PROFILES_ACTIVE=prod --service-account java-app-sa@PROJECT_ID.iam.gserviceaccount.com --no-allow-unauthenticated # require IAM token or API key

# Connect to VPC (access Cloud SQL in private VPC)
gcloud run services update java-api --vpc-connector vpc-connector-central --vpc-egress all-traffic
```

Cloud SQL Auth Proxy (Kubernetes):

```
# Sidecar in pod spec
spec:
  containers:
    - name: java-app
      image: java-api:latest
      env:
        - name: DB_URL
          value: "jdbc:postgresql://127.0.0.1:5432/javadb" # connect to proxy
    - name: cloud-sql-proxy
      image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2
      args:
        - "--port=5432"
        - "PROJECT_ID:us-central1:java-db"
      securityContext:
        runAsNonRoot: true
```

Interview Q&A – GCP Core Services for Java

Q1. What is the difference between GKE Standard and Autopilot?

Standard: you manage node pools (machine types, scaling, OS patching). Full control, SSH to nodes, spot instances. Bill per node. Autopilot: Google manages all nodes. You only specify pod CPU/memory. Bill per pod. Enforces security constraints (no privileged pods, no host networking). Better for teams wanting managed infrastructure; Standard for teams needing custom node configs, GPU workloads, or spot instance cost savings.

Q2. How does Workload Identity eliminate credentials in GKE pods?

Workload Identity maps a Kubernetes ServiceAccount to a GCP Service Account. Pods using the K8s SA automatically get GCP credentials via the GKE metadata server -- no JSON key files, no environment variables with secrets. GCP SDK auto-discovers credentials from the metadata endpoint. If pod is compromised, credentials are scoped to the service account's IAM permissions and expire automatically. Equivalent to AWS IRSA.

Q3. When should you use Cloud Run vs GKE for Java?

Cloud Run: simple HTTP services, variable traffic (scale to 0), fast deployment, no infrastructure ops. Limitations: no persistent storage, max 60-min request timeout, stateless only, no DaemonSets/CronJobs/StatefulSets. GKE: complex multi-container apps, stateful workloads (Kafka, databases), custom scheduling, WebSockets, gRPC streaming, cost optimisation with spot nodes. Spring Boot microservices: Cloud Run for simple services, GKE for complex stateful architectures.

Q4. What is Cloud SQL Auth Proxy and why use it instead of direct connection?

Cloud SQL Auth Proxy: process that creates a secure tunnel to Cloud SQL using IAM auth. Benefits: no IP allowlisting (proxy handles auth), encrypted connection automatically, no SSL certificate management, rotates credentials transparently. Run as sidecar in Kubernetes pods or as binary on VMs. Java app connects to 127.0.0.1:5432 -- proxy forwards to Cloud SQL over encrypted IAM-authenticated tunnel.

Q5. How does Cloud Run handle Java cold starts?

Cloud Run cold start = container start + JVM init + Spring context init. Solutions: min-instances=1 keeps one instance warm (small cost, eliminates cold start for production). Spring Boot optimisations: -Dspring.jmx.enabled=false, lazy loading (spring.main.lazy-initialization=true). Native image (GraalVM): near-instant startup but requires ahead-of-time compilation. Java 21 + CRaC (Project Leyden): checkpoint restore similar to Lambda SnapStart.

Q6. What is Binary Authorization in GKE?

Binary Authorization requires container images to be signed by trusted authorities before deployment. Policy: require signature from CI/CD pipeline's service account. Prevents: deploying unsigned images, images from untrusted registries, images that failed security scans. Setup: GCP Cloud Build signs images during CI, Binary Authorization policy enforces signature in GKE admission. Breaks supply chain attacks where attacker pushes malicious image to registry.

Q7. How does Cloud Armor protect Java backends on GCP?

Cloud Armor attaches to Google Cloud Load Balancer (HTTP(S) LB). Features: OWASP Top 10 preconfigured rules, custom WAF rules (SQL injection patterns, XSS), rate limiting per IP, geo-based blocking (block entire countries), adaptive protection (ML-based DDoS detection). Preview mode: log without blocking (tune rules). Blocking mode: active protection. Works at Google's edge -- malicious traffic blocked before reaching GKE/Cloud Run Java backends.

Q8. What is GKE's vertical pod autoscaling (VPA)?

VPA: Kubernetes Vertical Pod Autoscaler recommends or auto-sets CPU/memory requests based on actual usage. Modes: Off (just recommendations), Initial (set on pod creation), Auto (update running pods -- causes restarts). Useful for right-sizing Java pods: Spring Boot apps often over-provisioned. VPA recommendation: review for 1-2 weeks, then set requests manually. Caution: VPA Auto mode + HPA scaling on CPU can conflict.

Q9. How do you configure Cloud Run for VPC connectivity?

Cloud Run services run outside your VPC by default. VPC connector: --vpc-connector allows Cloud Run to access private VPC resources (Cloud SQL private IP, Redis on GCE). --vpc-egress all-traffic: all outbound traffic routed via VPC (for firewall control). --vpc-egress private-ranges-only: only RFC1918 traffic via VPC, internet direct. Direct VPC egress (newer): no connector needed, pods get VPC IP directly.

Q10. What is Artifact Registry and how does it secure Java CI/CD?

Artifact Registry stores Docker images, Maven/Gradle packages, npm packages with IAM-based access control (no public access by default). Pull from GKE: Workload Identity grants node SA roles/artifactregistry.reader. Push from Cloud Build: build SA gets

roles/artifactregistry.writer. Vulnerability scanning: Container Analysis scans images on push, blocks deployment of CRITICAL CVEs when combined with Binary Authorization. Regional: images stored in same region as GKE cluster for low-latency pulls.